

软件新技术专题 · 课程大作业

AI 辅助开发实践报告

高校实验室资产与报修管理平台

项目名称	高校实验室资产与报修管理平台
团队编号	第 6 组
团队成员	王旻辉、巴力江·托力肯别克、廖煌、迪丽热巴·阿布都西克尔
班级 / 学号	软件工程 2023-2 / 2311231056（其余 3 人学号 _____）
二级学院 / 专业	_____
授课教师	谭清萱
版本号	V1.0
日期	_____
编写人 / 审核人	_____ / _____

本文件是源稿（Markdown）。定稿流程：AI 出初稿 → 人工改写表述（防查重）→ 回填 → 导出 PDF（须嵌入字体）。

封面字段一律照 `docs/大作业/封面署名信息.md` 填写，不要从别处抄。

修订历史

版本	日期	修订内容	修订人
V0.1	_____	初稿完成（AI 辅助生成；事实取自 <code>docs/大作业/AI留痕/</code> 全量留痕）	_____
V1.0	_____	人工改写表述、回填 、定稿	_____

编写说明与 AI 使用声明

本文档记录本项目使用 AI 工具的方式、产出与人工把关情况。事实取自仓库内 `docs/大作业/AI留痕/` 的原始记录，未作事后美化。

章节	生成方式	说明
1 工具说明与使用方式	AI 生成初稿 → 待人工改写	工具清单按实际使用登记，角色划分引自项目公共记忆
2 三处完整提示词与生成结果	AI 整理 → 待人工改写	提示词为原文或按要点还原，结果可复核
3 人工审查与代码修改	全部为人工审查结论	取自留痕的「人工修改点」栏，含失败现场
4 效率对比	AI 整理 → 待人工改写	预估口径已写明，取值偏保守
5 反思、局限与改进	人工撰写	局限照实列出，未粉饰

防查重提示（提交前请删除本条）：第 2 章的提示词原文与项目内部的角色说明、任务卡存在同源风险。

定稿时须人工确认：本报告展示的提示词均为实际使用过的原始指令，不因查重而删改；

若整包重复率逼近红线，应改写第 1 / 4 / 5 章的叙述性段落，不动提示词原文。

1 工具说明与使用方式

1.1 使用的 AI 工具

类别	工具	在本项目中的作用
大语言模型	DeepSeek V4 Flash	代码生成与重构、SQL 与 Mapper 补全、异常排查思路生成
大语言模型	GLM 5.3 Flash	长上下文整体阅读（读完整目录后给结论）、文档初稿的语言组织
开发环境	VS Code	代码阅读与编辑、终端执行、版本控制；AI 产出的改动均在此逐条复核
智能平台	WorkBuddy	角色化会话驱动全流程：读项目记忆 → 领任务卡 → 改代码 / 写文档 → 跑验收命令 → 回写留痕

配套工具链（非 AI）：Chrome 152 + DevTools Protocol 用于界面截图与元素测量（本机无 Playwright 且离线不可安装，改用自写 Node 脚本直驱）；Maven 3.9.4 离线仓库 + JUnit 5 + Mockito + H2 1.4.199 用于构建与测试；MySQL 8、Redis 为运行时依赖。

1.2 AI 承担的角色与边界

AI 在本项目中不是“单点问答工具”，而是按六个角色并行接入，每个角色都有明确的不做什么：

#	角色	明确不做
1	产品负责人（公共记忆、需求基线、台账）	不替人做选题与范围的最终决策
2	前端 UI 负责人（业务页视觉统一、边界态兜底）	不改若依原生页面、不改首页视觉基线、不加新依赖
3	编码负责人（后端业务包、控制器、流水线、设计模式）	无任务卡不改生产代码；不擅自动远程仓库

#	角色	明确不做
4	视觉设计负责人（演示文稿版式、矢量图、海报）	不自行编写技术内容；不改源码去凑一张好看的截图
5	文档负责人（五份交付文档，含本报告）	不改任何代码；不编数据，拿不到就标
6	测试负责人（单元测试、集成测试、测试文档数字）	不改生产代码；测出的缺陷登记后交编码负责人修

这条分工有一个直接效果：写测试的人与写被测代码的人分开，从机制上避免了“自己测自己”。

1.3 留痕机制 —— 本报告的事实来源

项目给 AI 协作定了一条规矩：先记账，再干活；每完成一张任务卡就留一份痕，没编号的修复等于没发生。留痕目录共 11 份记录，每份固定七栏：基本信息、完整提示词、生成结果、人工修改点、效率对比、截图清单、遗留问题，分计划内 7 份（建表脚本；单测 30 与集成 7，加固轮后合计 49；前端 UI 统一与目视验收；五份文档撰写；演示文稿与视觉规范三轮）、计划外 2 份（一次“查缺补漏”查出的两个权限缺陷与一次流程偏移；演示路径冻结前的端到端保障，两阶段共 73 条接口采集）、以及 1 份跨角色文件（49 个用例的数字、命令、截图与业务规则追溯矩阵）。其中人工修改点是最值钱的一栏 —— 本报告第 3 章的内容全部取自它。

2 三处完整提示词与生成结果

三处分别对应后端缺陷修复、测试工程、文档工具链，覆盖本项目 AI 协作的三类主要场景。提示词按会话原文呈现（其中两条为按要点逐段还原，引号内为原话），未作润色。

2.1 提示词一 | 一次计划外的一致性巡检

背景：项目刚经历过一次“自定义代码从若依原生包迁出”的大重构与一轮角色白名单修复，这类改动最典型的后遗症是同一条权限在多个位置口径不同步（后端注解、菜单权限、角色授予、前端指令、行级过滤）。

完整提示词：

查缺补漏上

你查看了公共的memory了吗，你的角色就是负责代码的编写，在公共的资料中填入你的身份和你的工作，在完成要及时的记录和确认任务是否对应要求，其余在完善之后再说

这两条短到看起来不像“提示词”，却是本项目产出效率最高的一次交互。原因是项目已沉淀了一份公共记忆文件，里面写着角色边界、任务卡清单、已知偏移编号与验收命令；AI 在会话开始时就通读它，因此“查缺补漏”四个字落地时，它自己去把五个环节检索出来并交叉比对了，而不是反问“补什么”。第二条提示词纠正的不是代码，而是流程（见 3.2 案例二）。

生成结果：

项	内容
文件与规模	12 个（后端 8 + 前端 2 + 测试 1 + 文档 1）；新增 / 改写约 120 行，净删约 40 行；返工 2 次（测试期望值 1 次、流程纠偏 1 次）
验收输出	后端 Tests run: 7, Failures: 0, Errors: 0, Skipped: 0 + BUILD SUCCESS（1 分 39 秒）；前端生产构建 EXIT=0, 2545 modules transformed
查出的问题	2 个越权 / 可见性缺陷（登记为偏移编号），1 次流程偏移

2.2 提示词二 | 让“跑不起来”的看板查询在测试库上跑起来

背景：看板模块的“近 7 天趋势”两条查询用了 MySQL 的 date_format() 与 date_sub()，而测试用的 H2 内存库没有这两个函数。首轮结论是“看板无法覆盖，只能记为容忍”。

指令要点：

【开工指令 · 测试负责 · 第二轮】（T1 / T2 已完成，进入「交付 + 加固」）

先读 AGENTS.md + 5.1 节「测试负责」那一行 + T1/T2 卡，再核一遍现状。

【本阶段第一件事：交付】整理一份测试素材包交给「文档负责」：真实跑出来的用例数字、覆盖率证据、测试运行的命令行 + 截图、每个用例覆盖哪条业务规则（BR-01~BR-06 的对应关系）。

【本阶段第二件事：两处已知遗留】（择一或全做，都不许动 src/main）

- D-07 遗留：原 5 个状态机用例仍用 ReflectionTestUtils 打私有方法，可另写一组「经 updateLabRepair 公开入口」的状态机用例。
- D-19：看板查询用了 H2 不支持的 date_format()/date_sub()，已记录容忍。
★绝不许改生产 mapper XML 去迁就 H2，要补只能在测试侧注册同名自定义函数。
记得如实写明“看板未覆盖”——答辩若被问，含糊比承认更扣分。

【硬边界】可写 src/test/**；不改 src/main/**；不碰 docs/大作业/文档/**

【验收命令】bash .workbuddy/tools/mvnx.sh -o -B clean test

→ Tests run ≥37 且 Failures: 0。★ H2 必须写死 1.4.199

最关键的一句是“要补只能在测试侧注册同名自定义函数”—— 它把解法锁死在测试侧， 同时守住一条更硬的边界：不为迁就测试环境而修改生产 SQL。

生成结果：

项	内容
用例数	37 → 49（单元 30 → 33，集成 7 → 16），49 / 49 全绿（Failures / Errors / Skipped 均为 0）
新增覆盖与代码改动	看板 8 条 mapper 查询全部覆盖（7 个用例）；终态不可回退、不可复活各 1 例；状态机公开入口 3 例。生产代码零改动（git status 对 src/main 输出为空，生产 SQL 一字未改）
断言规模	断言 177 条、Mockito 交互校验 27 次；单次全量 1 分 38 秒，两次独立复跑一致

技术要点：H2 1.4.199 支持 create alias <函数名> for "<全限定类名>.<静态方法>"; 注册两个与 MySQL 同名的函数后， 生产 SQL 原文即可直接执行 —— 构建日志里的 Total: 3 正是用例期望的“今天 / 3 天前 / 6 天前”三个分组，窗口外的第 7 天被正确剔除。

2.3 提示词三 | 离线环境下的 Markdown → PDF 文档流水线

背景：五份交付文档要求为 PDF 且必须嵌入中文字体，但本机没有 pandoc、没有 LaTeX、没有 wkhtmltopdf，离线也装不上。

指令要点：

【开工指令 · 文档负责 · 第二轮】（2/5 初稿已成，进入「先打通 PDF」）

先读 AGENTS.md 全文 + 5.1 节「文档负责」那一行 + D1 卡，再核一遍现状。

【本阶段第一件事，优先级高于继续写：把 PDF 化这条路走通】

全仓现在 0 份 PDF — 9-20 是打包日，那天才开始试就爆了。

立刻拿 01 导一次 PDF、数页数（目标 8~12 页），一次性试出来：

- ①「md → PDF」的完整流程；② 页数能不能落进区间；③ 封面那些 [待填] 排出来是什么样。

导出方式自选，但要在留痕里写清用的是哪条路、命令是什么、踩了什么坑。

【打通之后，继续按已批准的写作顺序推进】

【硬边界】不改代码；不编数据，拿不到就标 [待填]；不碰 AGENTS.md 的 6 / 7 节台账。

【红线提醒】查重率 >30% 该项 0 分；每份开头必须有 AI 使用声明。

生成结果：

项	内容
自建工具与路径	<code>md2pdf.mjs</code> （Markdown → 打印用 HTML → 无头打印 PDF）、 <code>pdfinfo.mjs</code> （从 PDF 字节读页数 / 字体嵌入 / 文字可复制）、 <code>build.mjs</code> （批量编排 + 页数区间判定）。不引入任何依赖：复用项目既有的 CDP 手法，用 Node 22 自带的 <code>WebSocket</code> 与 <code>fetch</code> 直驱 Chrome 无头模式，调用 <code>Page.printToPDF</code>
产出与核验	需求规格说明书 8 页、软件架构设计文档 13 页，均落在任务书区间内；嵌入字体程序 5 / 8 个， <code>ToUnicode</code> 映射 76 / 87 个 → 中文不丢字、文字可复制；一条命令批量重建并自动给出 PASS / FAIL

这一条的实际价值不在“省了几小时排版”，而在于把打包日最大的风险提前拆掉了：文档源稿与 PDF 之间从“最后手动转一次”变成“随时可重建”。

3 人工审查与代码修改

本章取自各份留痕的「人工修改点」栏，累计记录 20 余处。以下 6 处按AI 的失效模式归类。

3.1 修改点汇总

#	场景	AI 原方案 → 问题所在 → 人工改为
1	测试垫片	<code>date_sub</code> 第二参数声明为 <code>int</code> → 报错信息里 H2 实际传进来的是字符串 <code>INTERVAL '6' DAY</code> → 声明为 <code>String</code> ，函数体内自行解析
2	测试基类	子类重复定义与基类同名的常量 → 子类定义静默遮蔽父类，基类改值后子类会带旧值静默通过 → 删除重复定义，改用基类常量
3	样式生效性	写了分页容器背景透明的规则 → 同权重下组件样式因异步 <code>chunk</code> 后加载而胜出，规则实际未生效 → 补 <code>!important</code> 并写明权重与加载顺序
4	验收脚本	用 <code>querySelector</code> 取滚动容器 → 同一页面有 3 组同名容器，抓到隐藏那一组，结论完全反转 → 改从可见元素出发 <code>closest()</code> 走祖先链

3.2 三个典型案例

案例一：AI 会自信地给出错误的具体数字。写“非法流转矩阵”时，AI 心算合法流转为 4 组，于是断言非法用例数应为 11；实际上 5 个状态的有序对总数是 25，扣掉 5 组幂等与 4 组合法，正确答案是 16。11 没有任何推导依据，是从“看起来能算出来”的地方凭印象落笔的。这个错误是测试自己报出来的——跑起来直接打印“应为 11，实际 16”并红灯。经验：若把断言写成 `assert checked > 0`，这个错误会一直躺在代码里，报告里“25 组矩阵全覆盖”就成了空话。断言的粒度决定了 AI 的错误能否被抓住。

案例二：AI 越界动手，被纠正的是流程而不是代码。巡检时 AI 直接改了生产代码下的 7 个 Java 文件与首页，改完只更新实现，没有在项目台账里认领任何任务卡。问题不在“改得对不对”：相关任务卡明确规定不改这些文件；更要紧的是本仓库可能被多个会话并行修改，跳过“认领”就等于放弃了唯一的撞车发现机制。处置是“修复保留、不回溯撤销”，但补齐三件事：登记偏移编号、补写进度台账、写下留痕记录，并把“无卡不改生产代码、紧急修复也必须先登记编号”写成明文规则。这条说明：AI 的越界往往不是技术错误，而是缺少流程约束——补边界比补代码更要紧。

案例三：AI 给出的“因果链”听起来对，但从未被验证。AI 为一行窄屏保护样式写的注释是“弹窗会超出视口，底部按钮既看不到也滚不到”。前半段对（横向确实溢出），后半段是错的：实测该弹层自带滚动容器，滚动 327px 后按钮位置完全可见——纵向问题与这次修复根本无关，两者被混为一谈。改成经得起复测的版本后，注释给出实测门槛（视口 1920 / 1366 / 900 下弹窗仍是声明宽度，800 下收成 768px，600 下收成 568px）。这是最典型的一类 AI 输出：格式工整、语气确定、却没有被验证过的因果解释。

4 效率对比

4.1 五个工作包的实测对比

工作包	纯人工（预估）	AI 辅助（实际）	返工	人工修改量
五层权限一致性巡检 + 2 个缺陷修复 + 状态机矩阵测试	6~8 h	约 2 h	2 次	3 处
测试加固（37 → 49 个用例，含 H2 方言垫片）	6~8 h	约 35 min	3 次	6 处
前端 UI 统一 + 真实环境目视验收 + 4 张截图	4~6 h	约 1.5 h	3 次	5 处
答辩演示文稿与视觉规范（3 轮，共 9 页版面）	12~16 h	约 3 h	5 次	8 处
离线 Markdown → PDF 文档流水线（自建工具）	8~10 h	约 2.5 h	4 次	4 处

预估口径：“纯人工”一列按本项目既往手写一个前后端模块（约 3.5 h）的耗时作参照推算，难点集中在技术不确定性上（H2 的方言能力边界、样式的加载顺序、无头浏览器的分页控制），这类“能不能做”的问题人工查文档最费时间。数字取值偏保守，不含流程纠偏与补写台账的时间。

从对比里能读出两条结论：第一，AI 省掉的是“写”，省不掉的是“验”——前端那一轮出题与改码约 0.4 h，验证占了 1.1 h，它抓出了“样式写在文件里但实际不生效”“验收脚本自己给错结论”“截图一调出来就看见的列宽问题”这三类静态检查永远发现不了的东西。第二，AI 的收益里有很大一块是“净删代码”——巡检一轮净删约 40 行（整段死代码 + 一个零调用的持久层方法）；死代码不删，下一个接手的人会以它们是活的。

5 反思、局限与改进

5.1 反复出现的四类失效模式

失效模式	表现	应对方式
自信的错误具体数字	矩阵用例数、色板色值数、页面计数都出过错，且都以陈述句给出	计数类数字一律用命令重算，并做两条独立路径的正交验证
未被验证的因果链	把“溢出”与“按钮不可达”写成因果，实际由不同机制负责	凡写入代码注释的因果，必须给出可复测的阈值
只覆盖正常路径	折叠态、窄屏、空数据、终态回退这类边界态反复遗漏	变更后必须在真实环境里逐项目视，不能只看静态代码
验证工具本身出错、倾向只报喜	验收脚本抓到错误元素后给出与事实相反的结论；交付文档初稿只想写“全绿、全覆盖”	工具输出与手工计算交叉比对，两条路径的口径差异能被解释才采信；不利事实照样写进文档

5.2 本报告承认的局限

1. 行覆盖率与分支覆盖率百分比拿不到真实值。离线仓库缺 JaCoCo 插件，缺插件就无法生成报告；引入它需改构建文件并联网下载，超出既定边界。因此以用例与业务规则的追溯矩阵作为需求维度的覆盖证据，并附断言数 177 条、交互校验 27 次。不编造一个百分比。2. 两处未直接验证的事项：“注入异常后断言双表回滚”因持久层组件是动态代理、注入类生效性存疑，目前由“失败时三处整体不变”的集成用例间接覆盖；两处边界态靠浏览器内注入合成数据补测（不写库、量完即移除）。补测方法均已在留痕中写明，可复核。3. AI 不承担决策。选题范围、口径取舍、验收结论均由人拍板，AI 的定位是执行、巡检与留痕。

5.3 已采取的改进

计数类数字设了一道关卡：动手前先重读项目台账原文，不信任会话上下文里的数字。起因是一次真实教训——任务下达时引用的用例数是 37，而权威台账早已更新为 49，旧数字正顺着下游产出复活。另外，留痕升格为强制项：每张任务卡完成即留一份记录，计划外的巡检也单独命名留痕；交付前对全部计数类数字（用例数、数据表数、接口数、文档页数）做一次全量回查。这条规矩的直接回报，就是本报告不需要靠回忆来写。